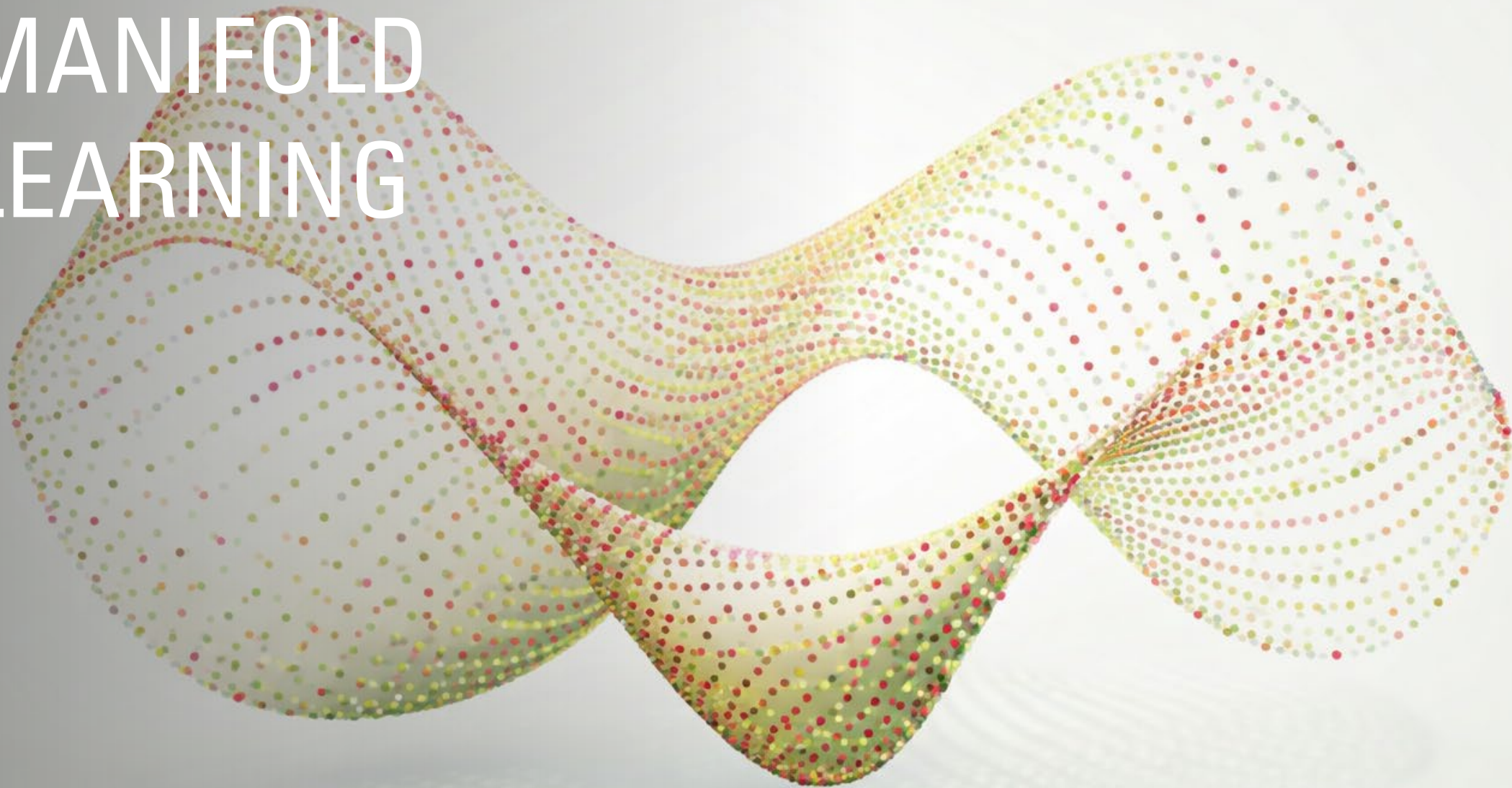
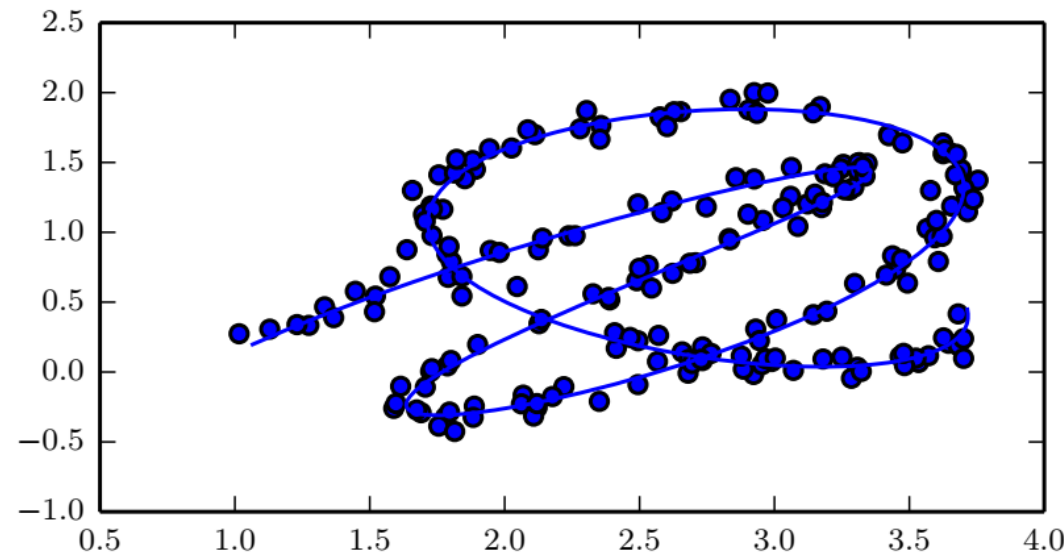

MANIFOLD LEARNING



MANIFOLD LEARNING

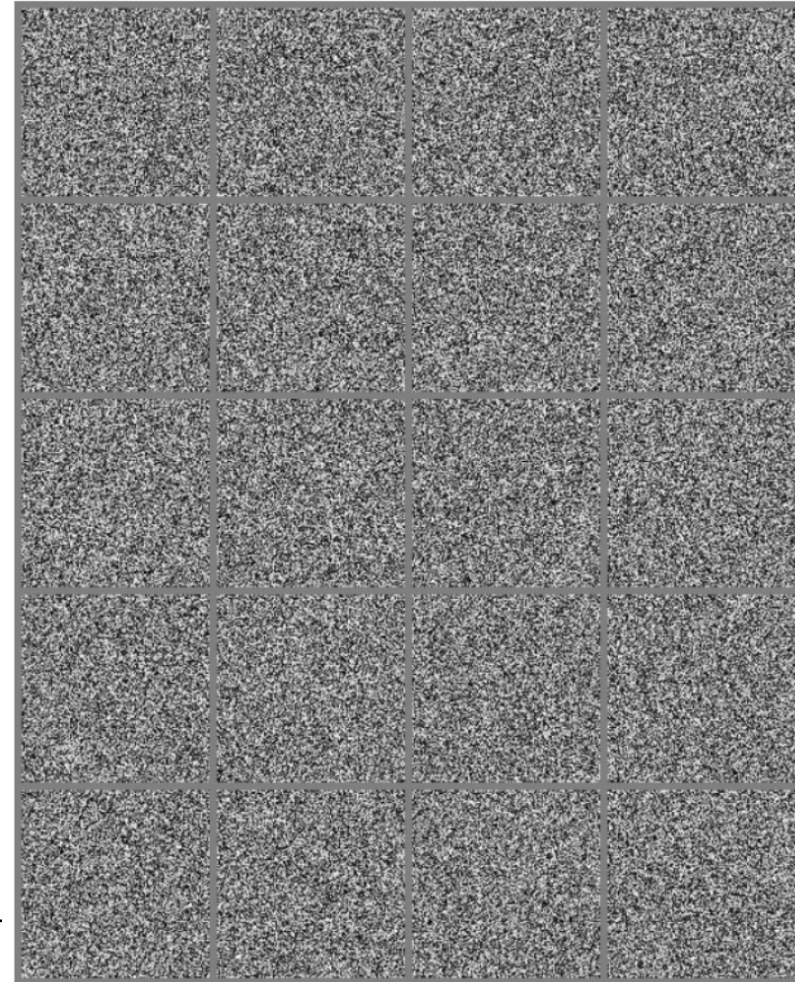
- Informally, a manifold in machine learning loosely means a connected set of points in high dimensions that can be approximated well by a small number of dimensions
 - Sometimes referred to as the **manifold assumption**
 - Simplifies the learning problem greatly and helps with the curse of dimensionality
- Is this a good assumption in machine learning?



Goodfellow et al.,
2016

THE MANIFOLD ASSUMPTION IN PRACTICE

- The probability distribution over images, text strings, and sounds that occur in real life is highly concentrated
- **Example:** randomly generated images (right) rarely (never?) look like real objects
- **Example:** picking random letters unlikely to give real text
- Is this sufficient for the manifold assumption?
- No, also need to establish that the examples we encounter are connected to each other by other examples



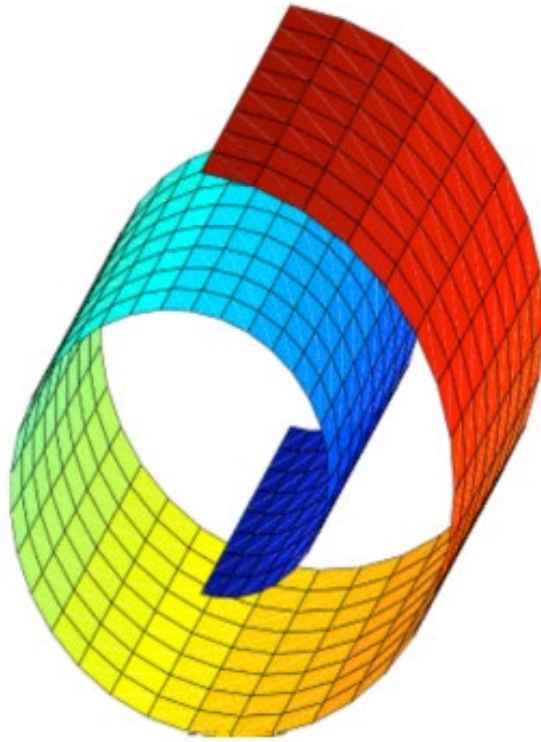


THE MANIFOLD ASSUMPTION IN PRACTICE

- In images, we can think of many possible transformations that trace out a manifold
 - Dim or brighten the lights, gradually move or rotate objects, etc.
- Multiple manifolds may be involved (e.g. human manifolds and cat manifolds)



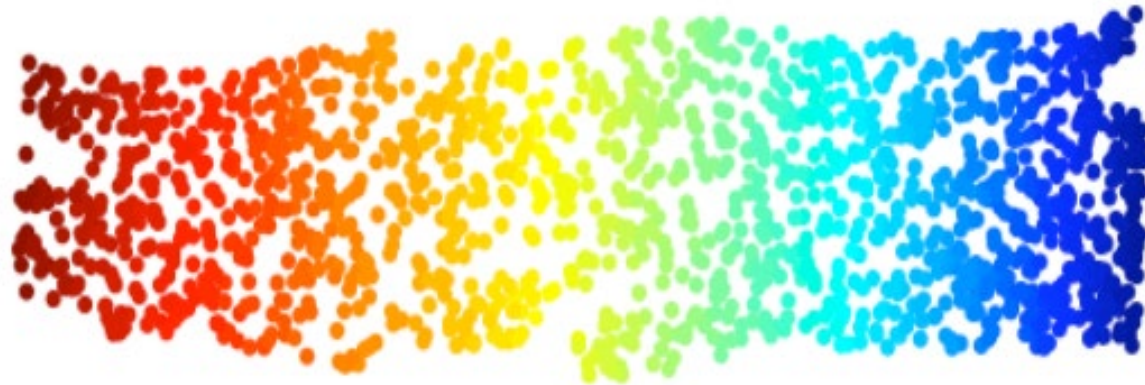
MANIFOLD LEARNING



MANIFOLD LEARNING

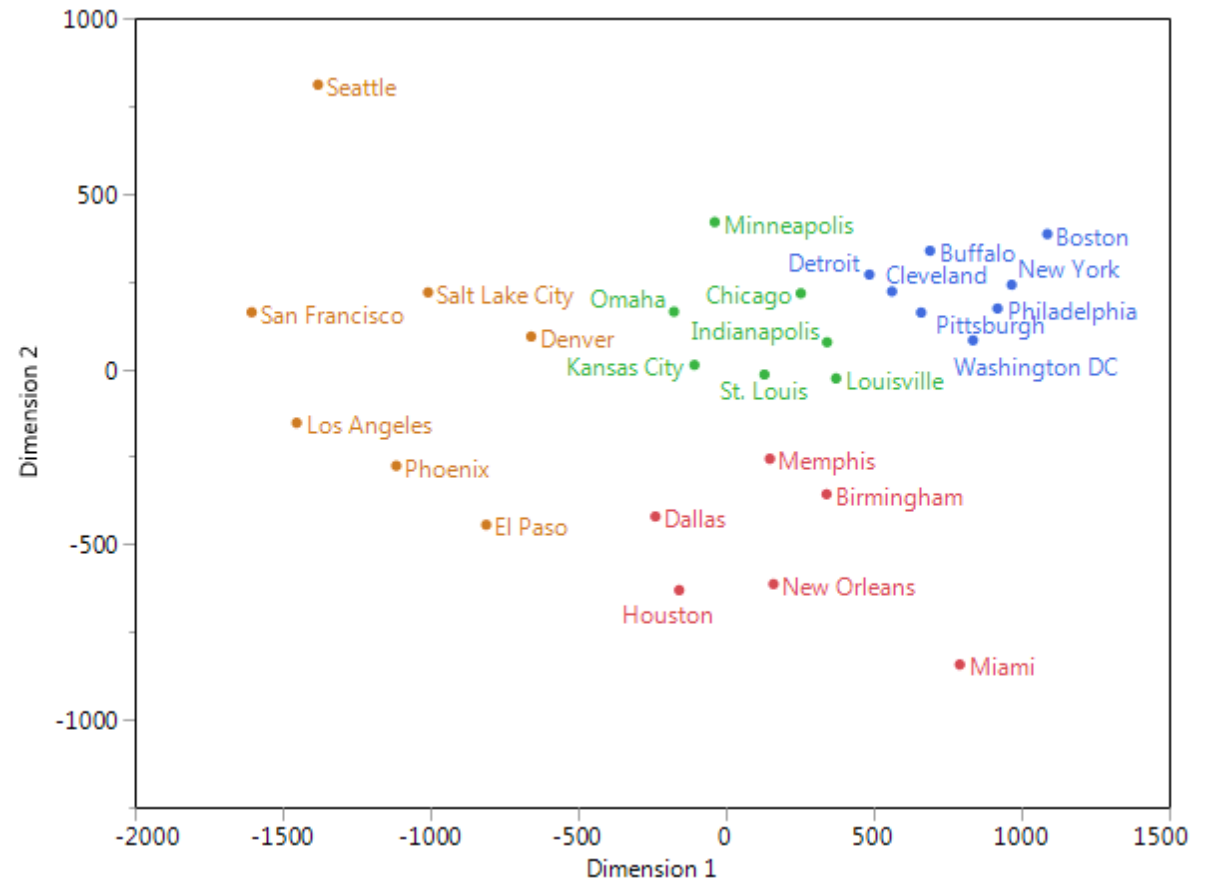


MANIFOLD LEARNING



MULTIDIMENSIONAL SCALING (MDS)

- MDS is a class of dimensionality reduction methods
- **Goal:** minimize the “difference” between corresponding distances in the high and low dimensional spaces
- **Example:** projecting cities on a 3D Earth to a 2D map



METRIC MDS

- Let D be a matrix of pairwise distances between n points in the high-dimensional space
- Metric MDS has a different “stress” function to minimize:

$$\text{Stress}(\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n) = \sqrt{\frac{\sum_{i,j} (D_{ij} - \|\hat{\mathbf{x}}_i - \hat{\mathbf{x}}_j\|)^2}{\sum_{i,j} D_{ij}^2}}$$

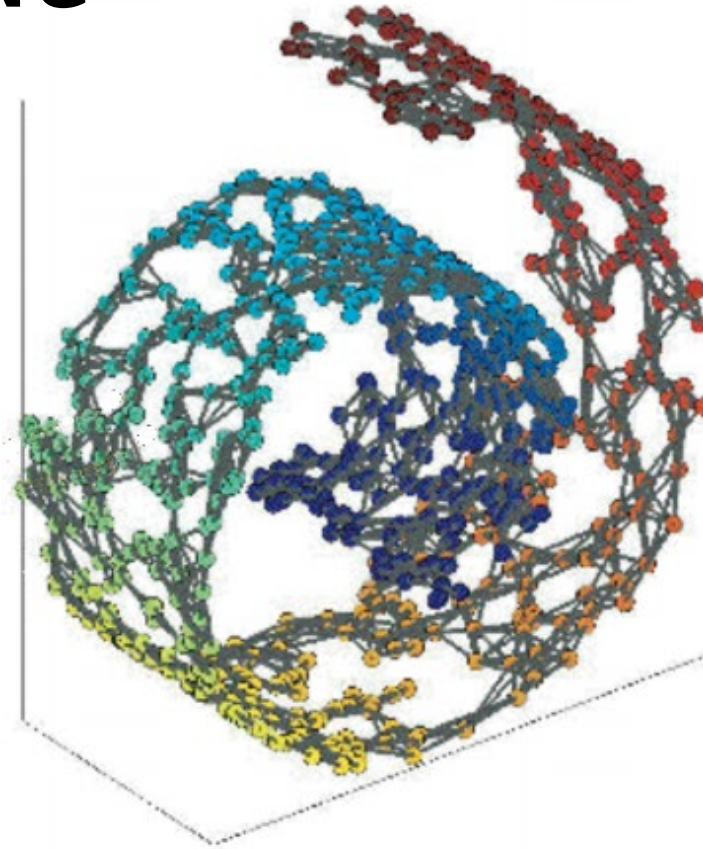
- Solving this requires an iterative approach (like gradient descent)
 - Generally gives better visualization than CMDS for PHATE
 - Nonmetric MDS also exists
 - Input “distances” do not need to be a true distance
-

MANIFOLD LEARNING

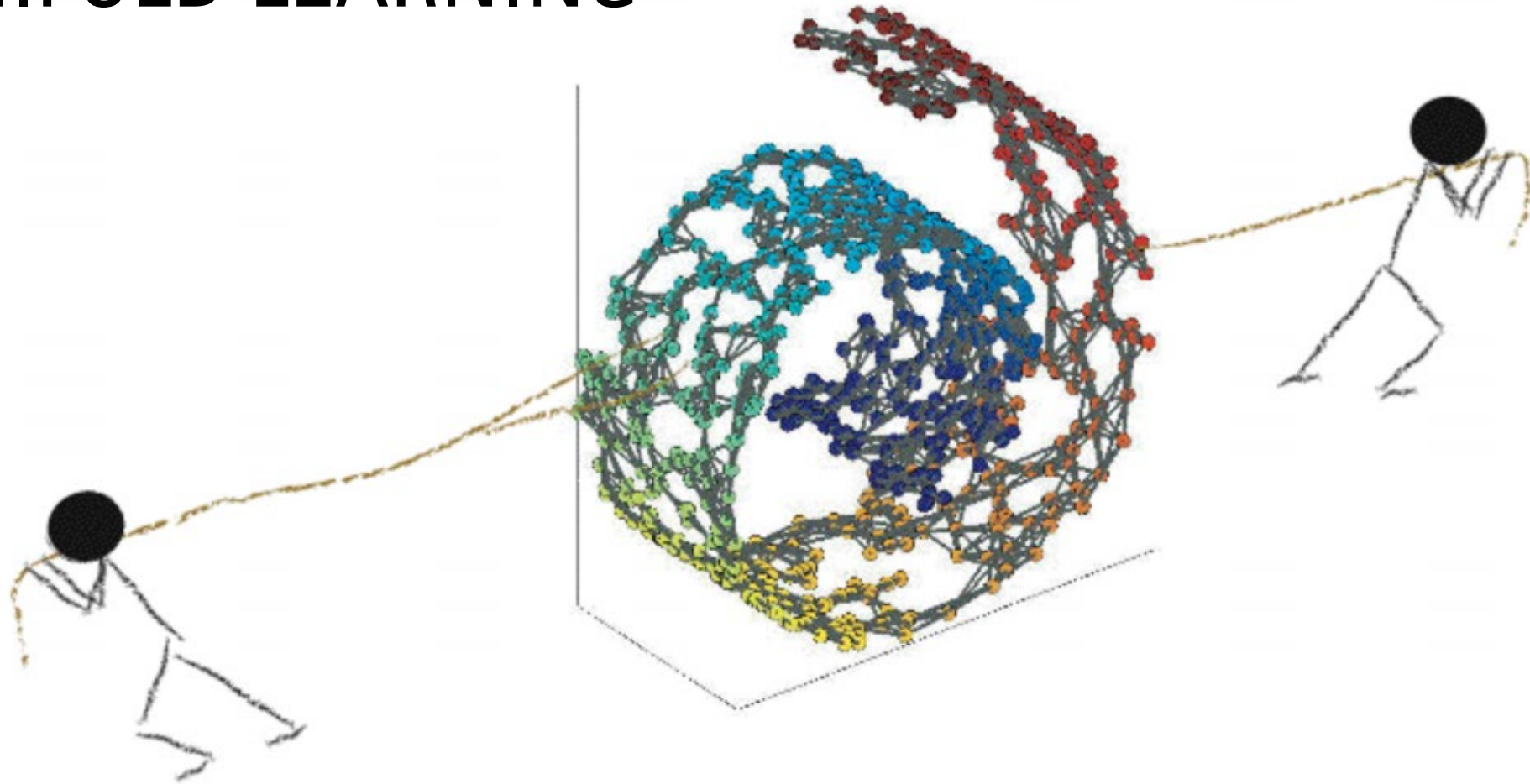
- Large Euclidean distances often uninformative
 - Exit the manifold
 - Also likely to suffer from the curse of dimensionality
- Thus MDS on Euclidean distances often isn't great
- **Solution:** input “better” distances
- We'll construct distances from a graph



MANIFOLD LEARNING



MANIFOLD LEARNING



SIMILARITY GRAPHS

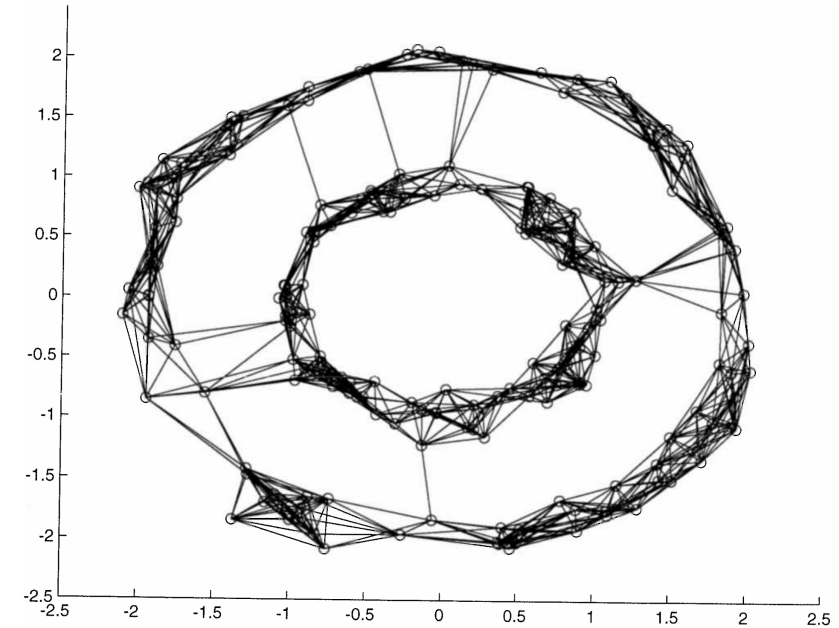
- Unlabeled data to be analyzed: $\mathbf{x}_1, \dots, \mathbf{x}_n$
- Assume relationships between data points are captured entirely by a *similarity graph*
- Similarity graphs are defined by an *affinity matrix*

$$W = \begin{bmatrix} w_{11} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{n1} & \dots & w_{nn} \end{bmatrix}$$

- Weights are nonnegative, symmetric:
$$w_{ij} \geq 0, \quad w_{ij} = w_{ji}$$
 - Associated graph: Nodes i and j are connected iff $w_{ij} > 0$
 - w_{ij} are the edge weights
-

SIMILARITY GRAPHS

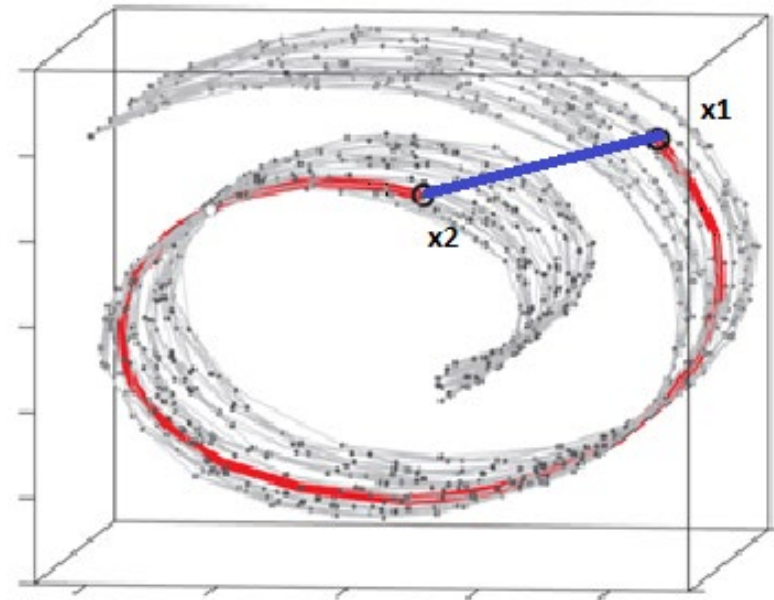
- Similarity graphs are defined by two things
- **Graph structure** (which edges are connected)
 - Examples
 - k -nn graph
 - ϵ -ball graph
 - Complete graph
- **Weights**
 - Examples
 - $w_{ij} = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$
 - $w_{ij} = \begin{cases} 1 & \text{if } \exists \text{ an edge between nodes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$



11-nn graph

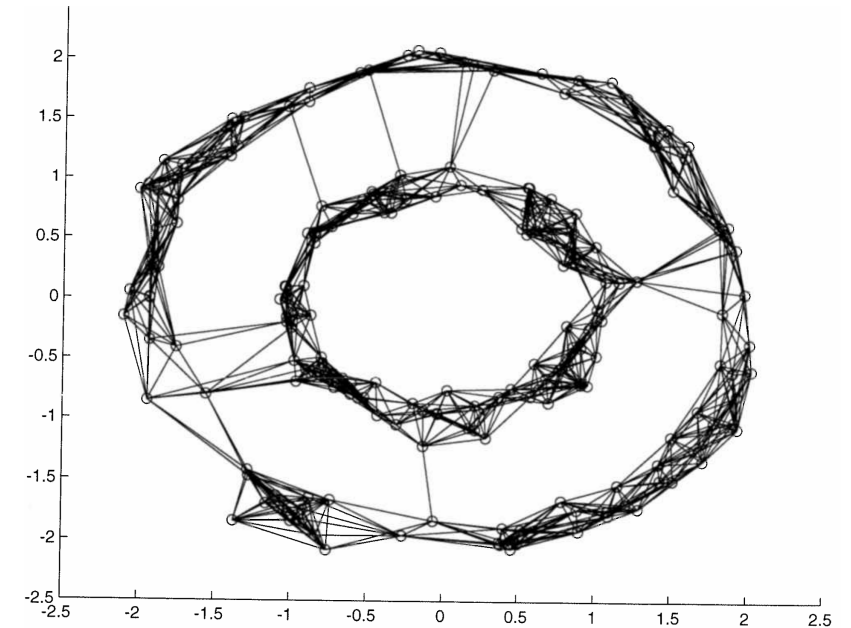
ISOMAP

1. Construct a neighborhood graph on the data
 - E.g. k -nn or ϵ -ball graph
2. Compute the shortest path along the graph between all pairs of points
 - The length of these paths is the geodesic distance
3. Embed the distances into low-dimensions using MDS



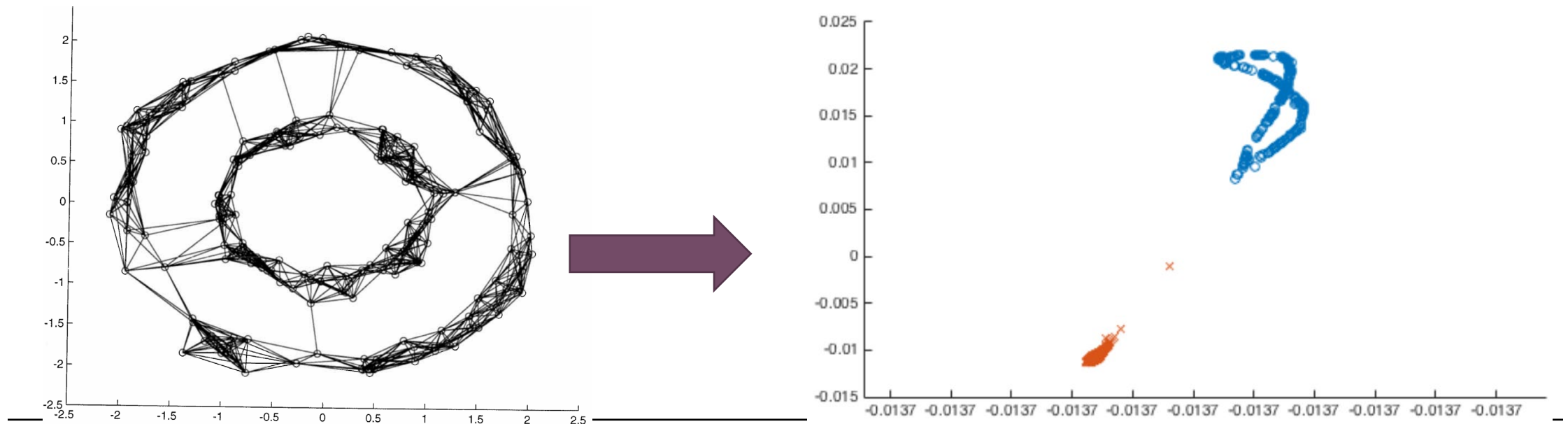
ISOMAP

- **Weakness:** ISOMAP is sensitive to spurious connections
 - Noise or too large of k may lead to exits from the manifold



LAPLACIAN EIGENMAPS

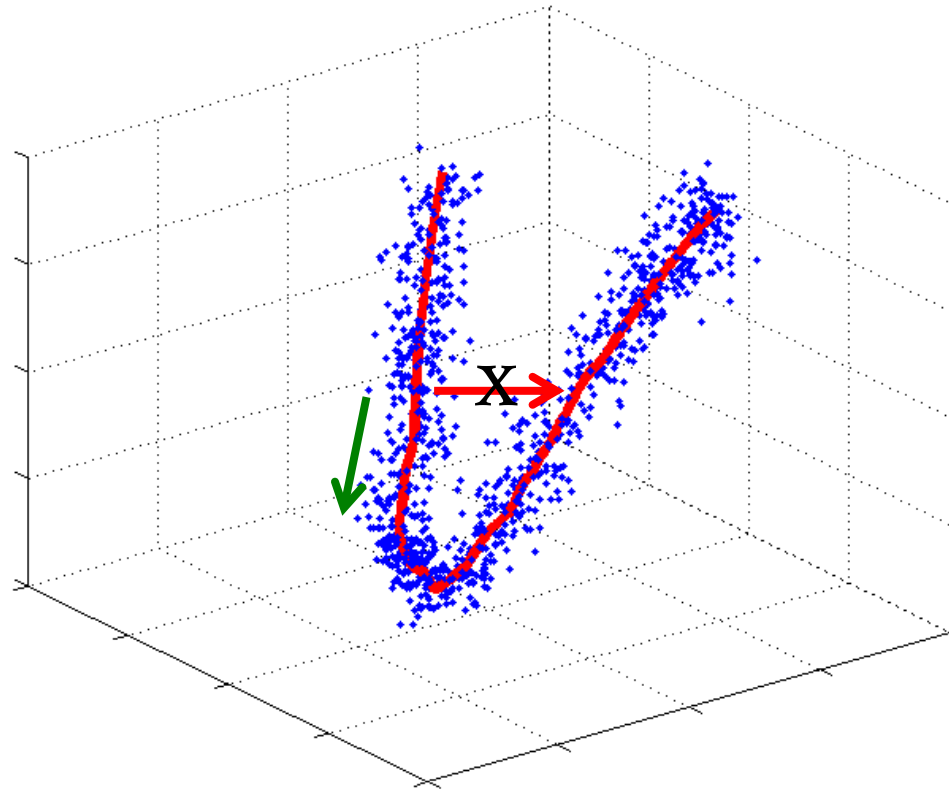
- Direct embedding of the graph structure
- Based on an eigendecomposition of the (normalized) graph Laplacian
- Works for clustering, but may not preserve local structure well



DIFFUSION MAPS

More robust than ISOMAP

CAPTURING



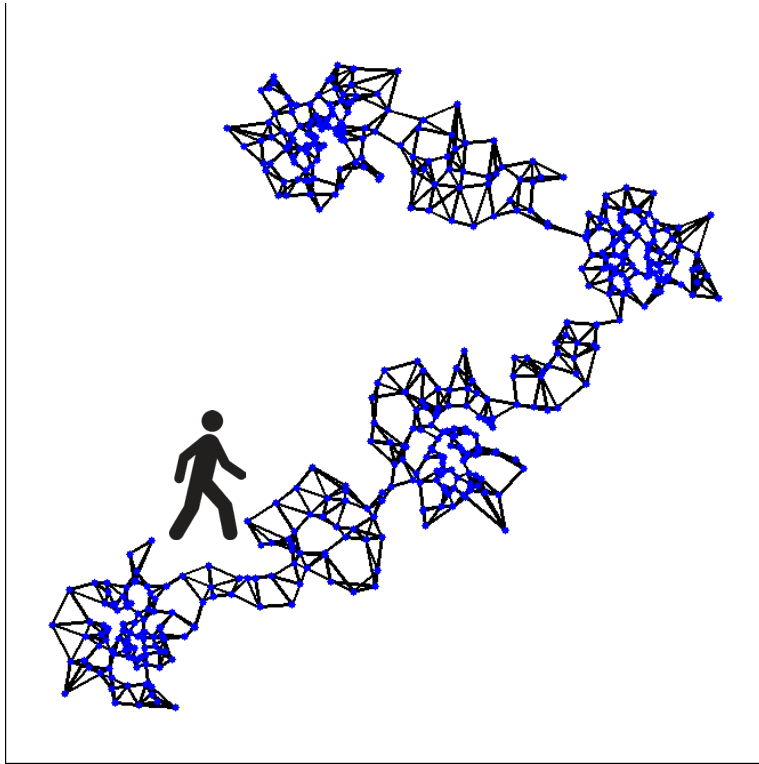
- Large Euclidean distance gives wrong global structure for visualizing
- Small Euclidean distances ok
 - Encodes local structure

FROM DISTANCES TO AFFINITIES

Affinity: a measure of similarity between points

- Step 1: Calculate all pairwise Euclidean distances
- Step 2: Convert the distances to pairwise affinities using a kernel function
 - E.g. the Gaussian kernel: $k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{\sigma^2}\right)$
 - Kernel function must be near zero for large distances and nonzero for small distances
- I.e. create a similarity graph
- Assume we have data points $\mathbf{x}_1, \dots, \mathbf{x}_n$

DIFFUSION OPERATOR



- Big Euclidean steps bad, likely to exit structure
- Small steps good, likely to stay within structure
- Shortest path (ISOMAP) is sensitive to outliers and spurious connections
- **Solution:** consider all possible paths via random walks
- Can do this computationally cheaply using diffusion

DIFFUSION OPERATOR

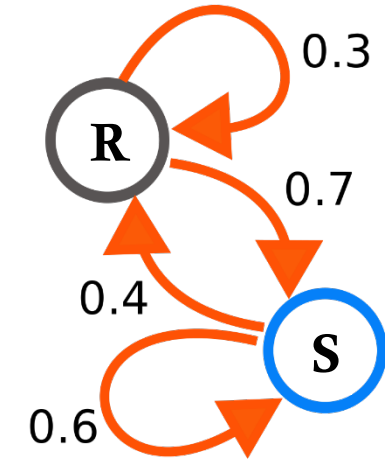
- Denote $K \in \mathbb{R}^{n \times n}$ as the pairwise affinity/kernel matrix
- Construct the degree matrix D
 - Sum the entries of each row and place along the diagonal
- Normalize K by D to obtain the diffusion operator P

$$P = D^{-1}K$$

- Properties of P :
 - A normalized Graph Laplacian
 - All entries are nonnegative
 - Not symmetric
 - Row stochastic (each row sums to 1)
 - Forms a Markov transition matrix
-

MARKOV PROCESSES

- Used to model a system's state over time
- Consists of a set of possible states and transition probabilities
- **Example:** suppose the weather is either sunny or rainy
 - Two states: sunny (S) and rainy (R)
 - If states are independent, then we flip a coin each day
 - Weather is usually NOT independent
 - Suppose we live in an area where it's uncommon for it to rain for multiple days
 - Model this with a Markov chain
 - Transition probabilities indicate how “sticky” each state is

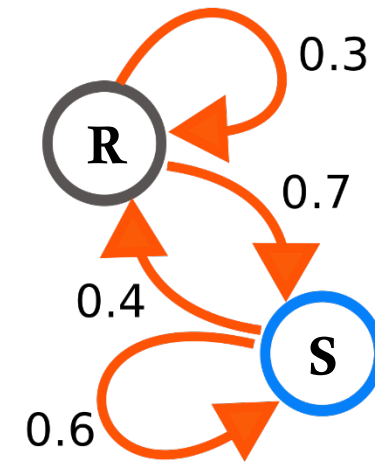


MARKOV PROCESSES

- **Example:** suppose the weather is either sunny or rainy
 - Two states: sunny (S) and rainy (R)
 - Probability transition matrix used to summarize the dynamics

$$P = \begin{matrix} & \begin{matrix} R & S \end{matrix} \\ \begin{matrix} R \\ S \end{matrix} & \begin{bmatrix} 0.3 & 0.7 \\ 0.4 & 0.6 \end{bmatrix} \end{matrix}$$

- P gives the probability of transitioning from one state to any other state in a single time step
 - Probability of transitioning from one state to any other in t time steps is obtained from P^t
- Markov assumption: future states depend only on the present state and are independent of past states
 - All the necessary “memory” is contained in the present state



STATIONARY DISTRIBUTION

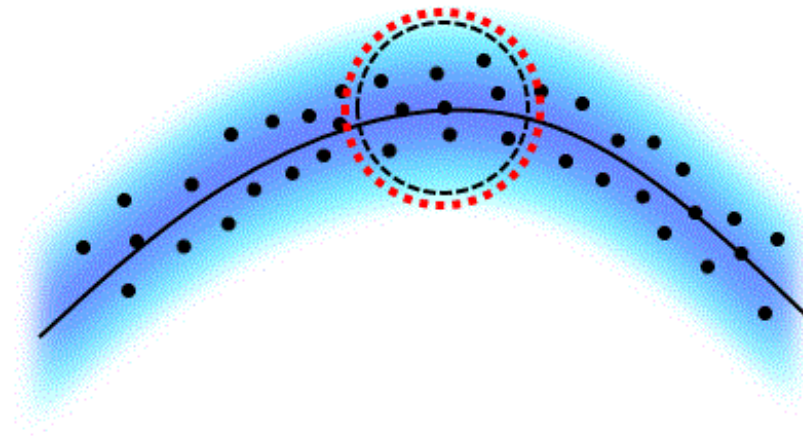
- Consider a probability distribution π as a row vector
 - Nonnegative elements that sum to 1

- π is a *stationary distribution* of the Markov chain if

$$\pi P = \pi$$

- π is unchanged by P
 - A stationary distribution represents the steady state conditions of the Markov chain
 - The probability you end up in each corresponding state after a long time (subject to initial conditions)
 - **Note:** π is an eigenvector of P with eigenvalue of 1
 - Can use the eigenvalue equation to find π
-

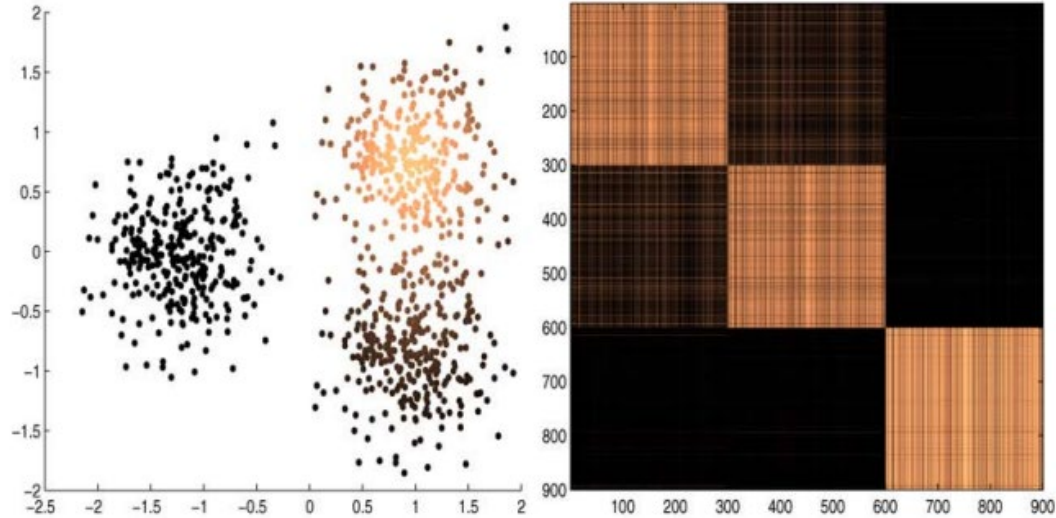
DIFFUSION DENOISES AND RECOVERS GLOBAL STRUCTURE



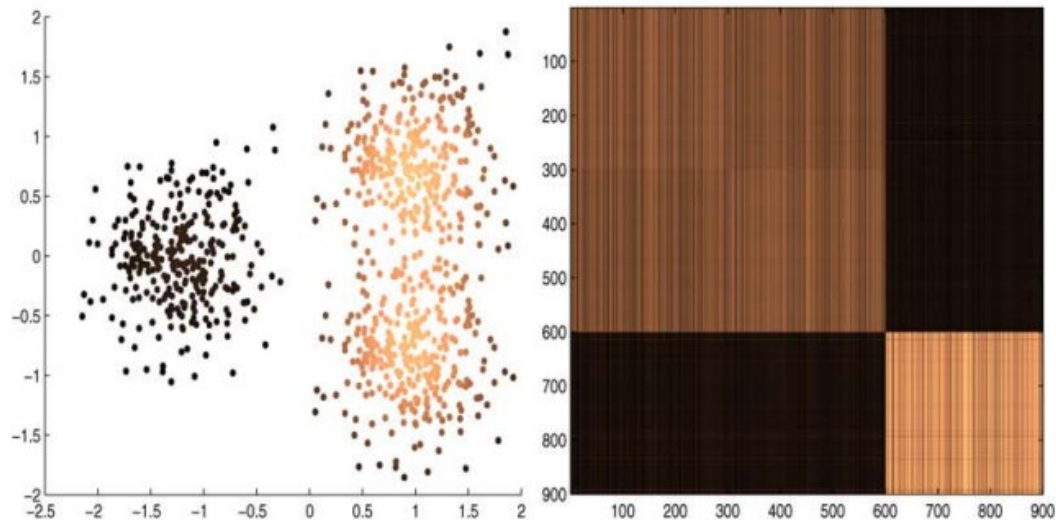
THE TIME SCALE t

- **Remark:** all eigenvalues of P are between 1 and -1
 - Powering a matrix is equivalent to powering the eigenvalues:
$$P^t = U\Lambda^t U^{-1}$$
 - Only the stationary distribution eigenvalues remain after large t
 - Thus too large of t can oversmooth the data
 - But too small of t doesn't learn the global structure
 - We'll see more on this when we talk about visualization and PHATE
-

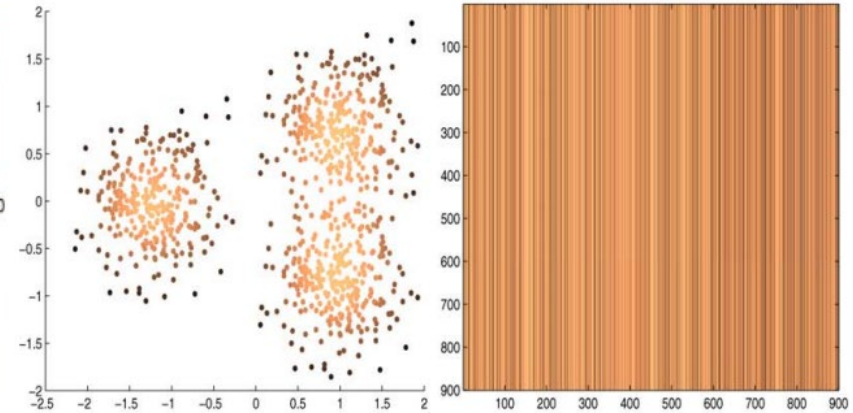
THE TIME SCALE t



(a) $t = 8$



(b) $t = 64$



(c) $t = 1024$

DIFFUSION MAPS

Algorithm for Diffusion Maps

1. Compute Euclidean distances
 2. Convert distances to local affinity matrix K
 - Classically, the Gaussian kernel was used
 3. Create degree matrix D from K (sum the rows of K and create a diagonal matrix D from the sums)
 4. Normalize: $M = D^{-1/2} K D^{-1/2}$
 5. Diffuse: M^t
 6. Obtain diffusion coordinates by performing eigenvalue decomposition of M^t
-

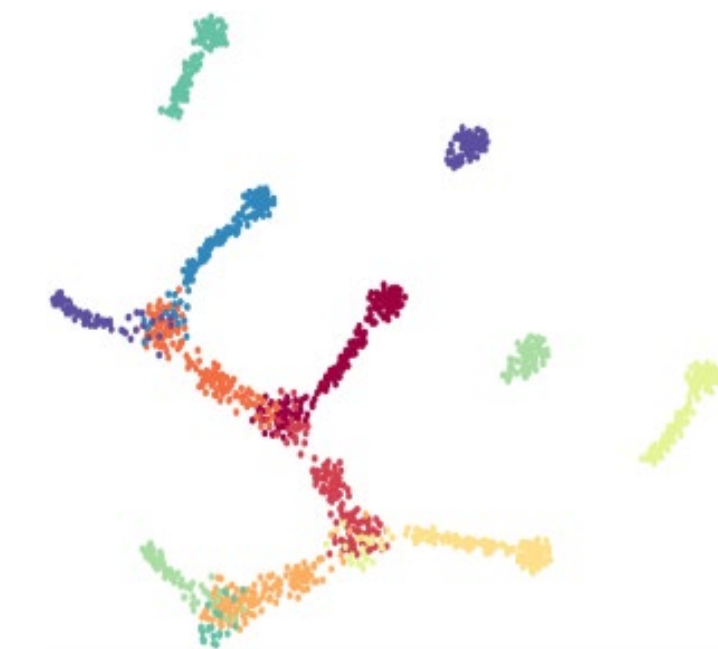
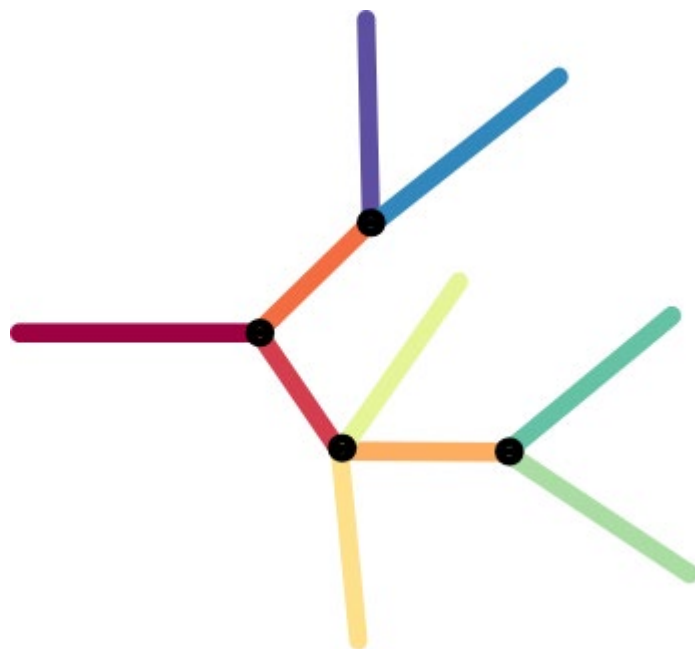
VISUALIZING STRUCTURE WITH MANIFOLDS



VISUALIZATION WITH T-SNE

Artificial Tree Data

t-SNE

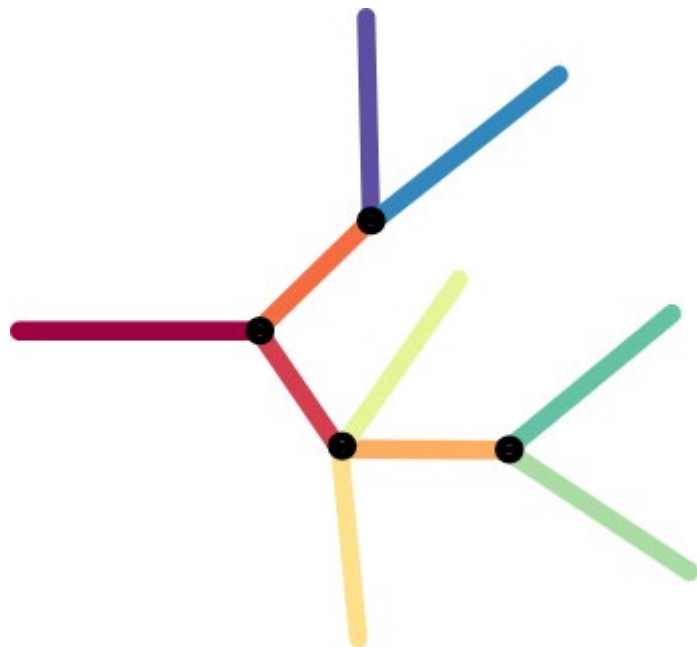


1400 points, 60 dimensions



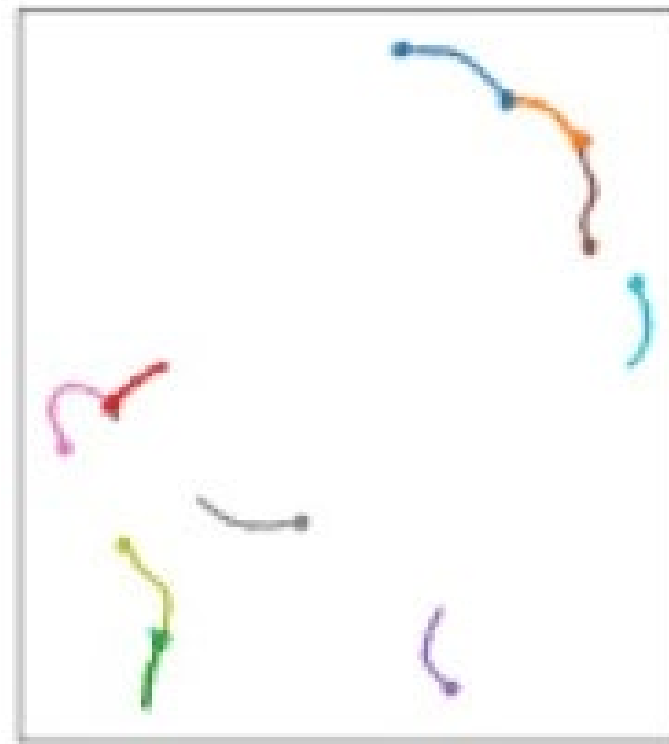
UMAP STILL FAILS AT GLOBAL STRUCTURE

Artificial Tree Data



1400 points, 60 dimensions

UMAP



BALANCING LOCAL AND GLOBAL INFORMATION

- T-SNE and UMAP are good for local structure
- PCA is good for global structure
- Diffusion maps (DM) does well at capturing both
- **Idea:** use DM to visualize data

t-SNE



UMAP



PCA



VISUALIZATION WITH DIFFUSION MAPS

- DM captures the information accurately and robustly, but not for visualization
- DM tends to place the structure in higher dimensions
- DM can also be unstable with boundaries and intersections

Artificial Tree Data



Diffusion Maps



1400 points, 60 dimensions

PHATE

(POTENTIAL OF HEAT-DIFFUSION FOR AFFINITY-BASED TRANSITION EMBEDDING)

Visualizing Structure and Transitions in High-Dimensional Biological Data (Nature Biotechnology, 2019)

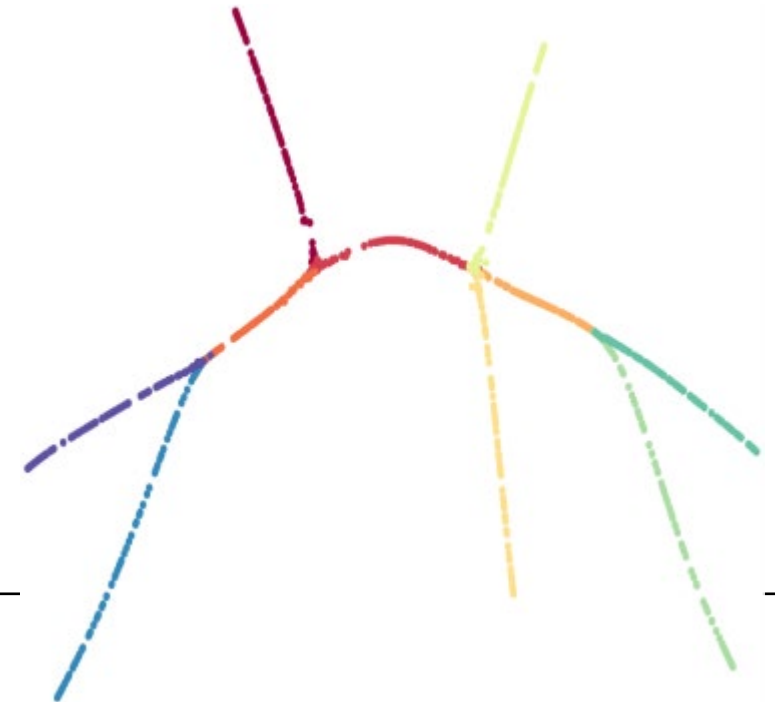
- A method for visualizing data built off of the diffusion framework

Artificial Tree Data

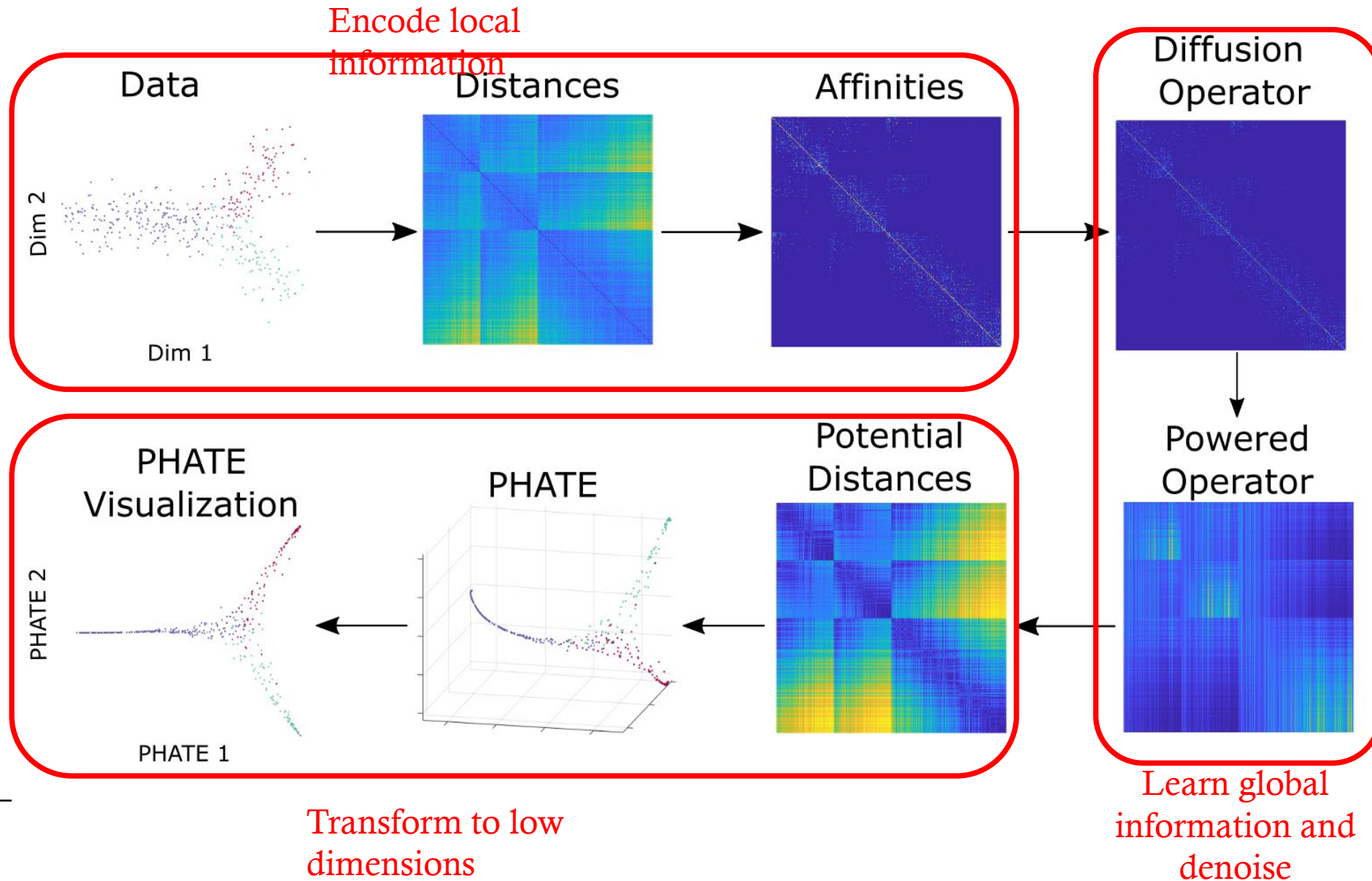


1400 points, 60 dimensions

PHATE



THE PHATE ALGORITHM



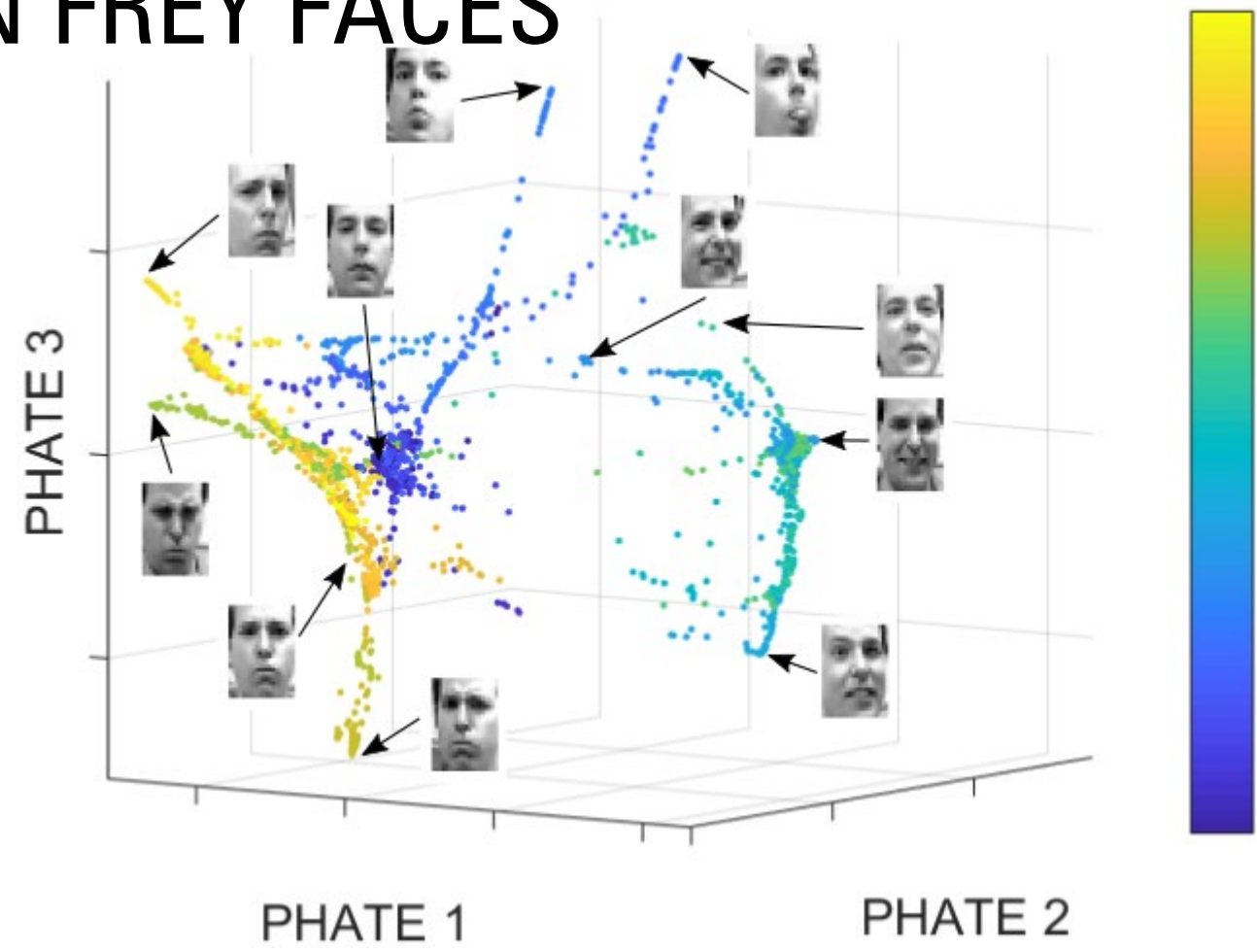
PHATE ON FREY FACES

- Frey Faces dataset
 - 1965 frames taken from a video of Brendan Frey making various faces in front of a camera
 - Resolution: 20x28
- Frames are given out of order to PHATE w/o information about sequential ordering





PHATE ON FREY FACES

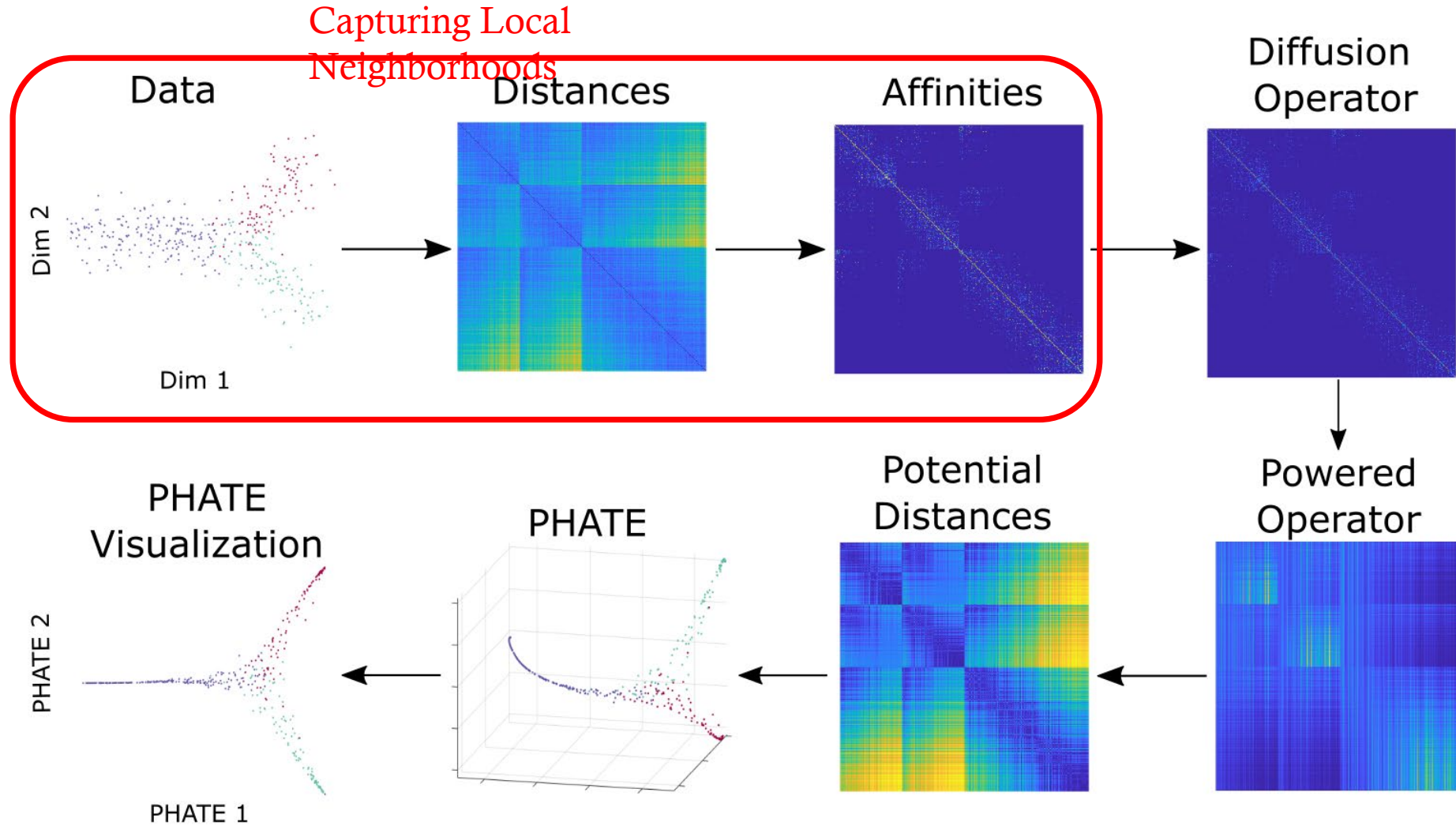




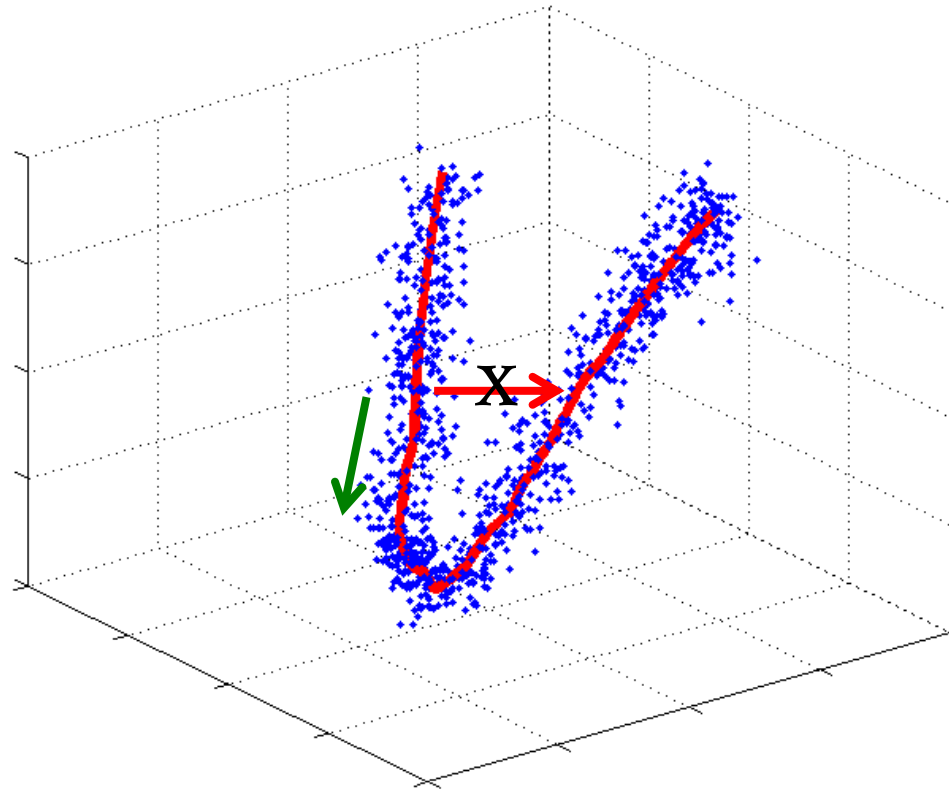
PHATE ON FREY FACES



THE PHATE ALGORITHM



CAPTURING



- Large Euclidean distance gives wrong global structure for visualizing
- Small Euclidean distances ok
 - Encodes local structure

FROM DISTANCES TO AFFINITIES

- Step 1: Calculate all pairwise Euclidean distances
 - Step 2: Convert the distances to pairwise affinities using a kernel function
 - E.g. the Gaussian kernel
 - Kernel function must be near zero for large distances and nonzero for small distances
 - Affinity: a measure of similarity between points
 - **Problem:** many data have both dense and sparse regions
 - A kernel bandwidth fixed for dense regions doesn't work well for sparse regions and vice versa
 - Idea: use locally adaptive bandwidth
-



CAPTURING LOCAL NEIGHBORHOODS

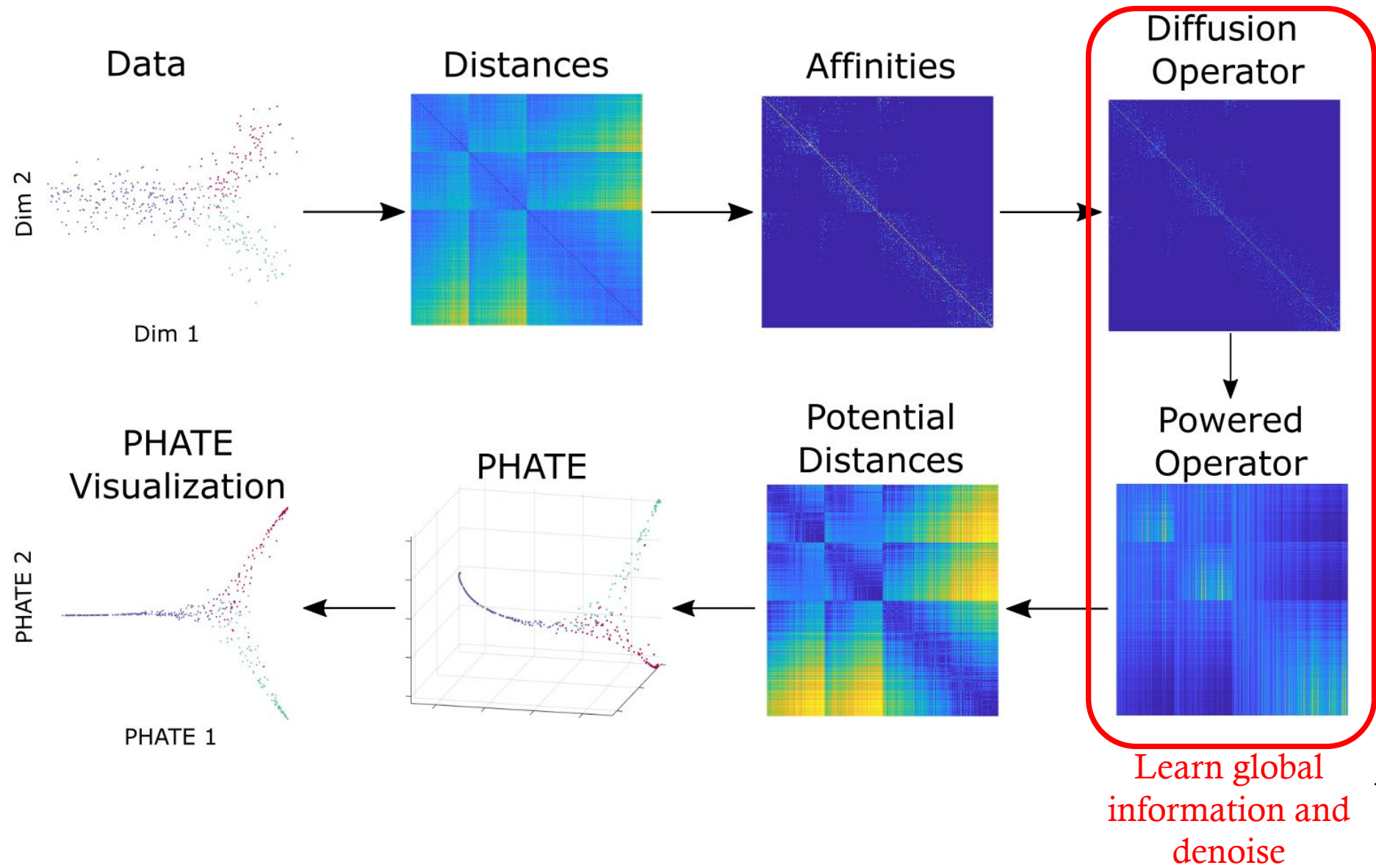
- Adaptive α -decaying kernel

$$\tilde{g}(x, y) = \exp \left(- \left(\frac{\|x - y\|}{\epsilon_x} \right)^\alpha \right), \rightarrow g(x, y) = \frac{\tilde{g}(x, y) + \tilde{g}(y, x)}{2}$$

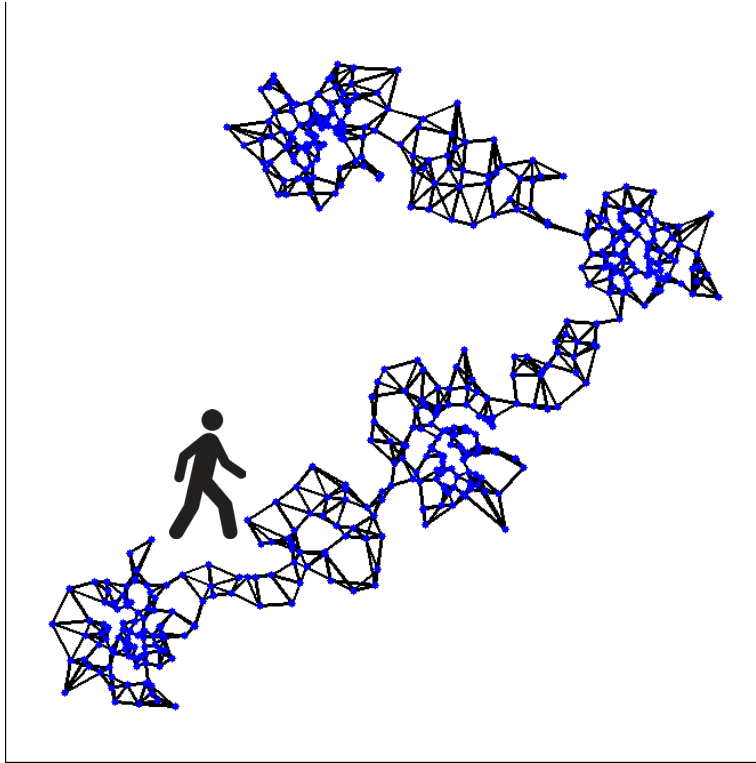
Where

- ϵ_x = distance from x to its k th nearest neighbor
 - α controls the decay rate of $\tilde{g}$
 - Provides a robust notion of adaptive locality
-

THE PHATE ALGORITHM



DIFFUSION DENOISES AND RECOVERS GLOBAL STRUCTURE



- Big Euclidean steps bad, likely to exit structure
- Small steps good, likely to stay within structure
- To learn global structure via small local steps we use random walk (diffusion)
 - Normalize affinity matrix to create Markov transition matrix (the diffusion operator) P (Coifman & Lafon, *ACHA*, 2006)
 - Power P by time step t

HOW MUCH DIFFUSION?

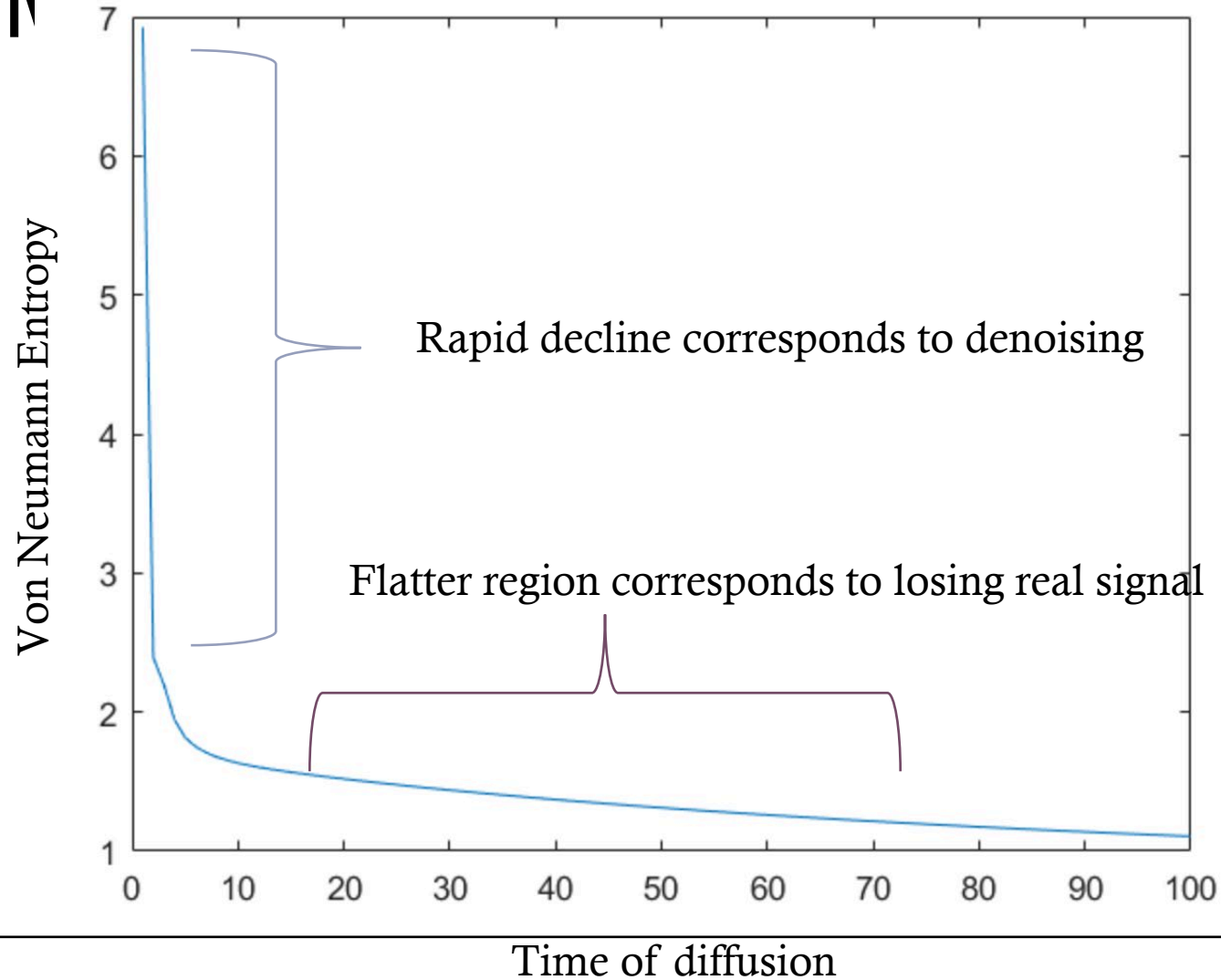
- Von Neumann entropy (von Neumann, 1932) of diffused operator P^t

$$VNE(P^t) = - \sum_j \eta_j \log \eta_j, \eta_j = \lambda_j^t / \|\lambda^t\|_1$$

Where $\lambda^t = \{\lambda_0^t, \lambda_1^t, \dots\}$ are the eigenvalues of P^t

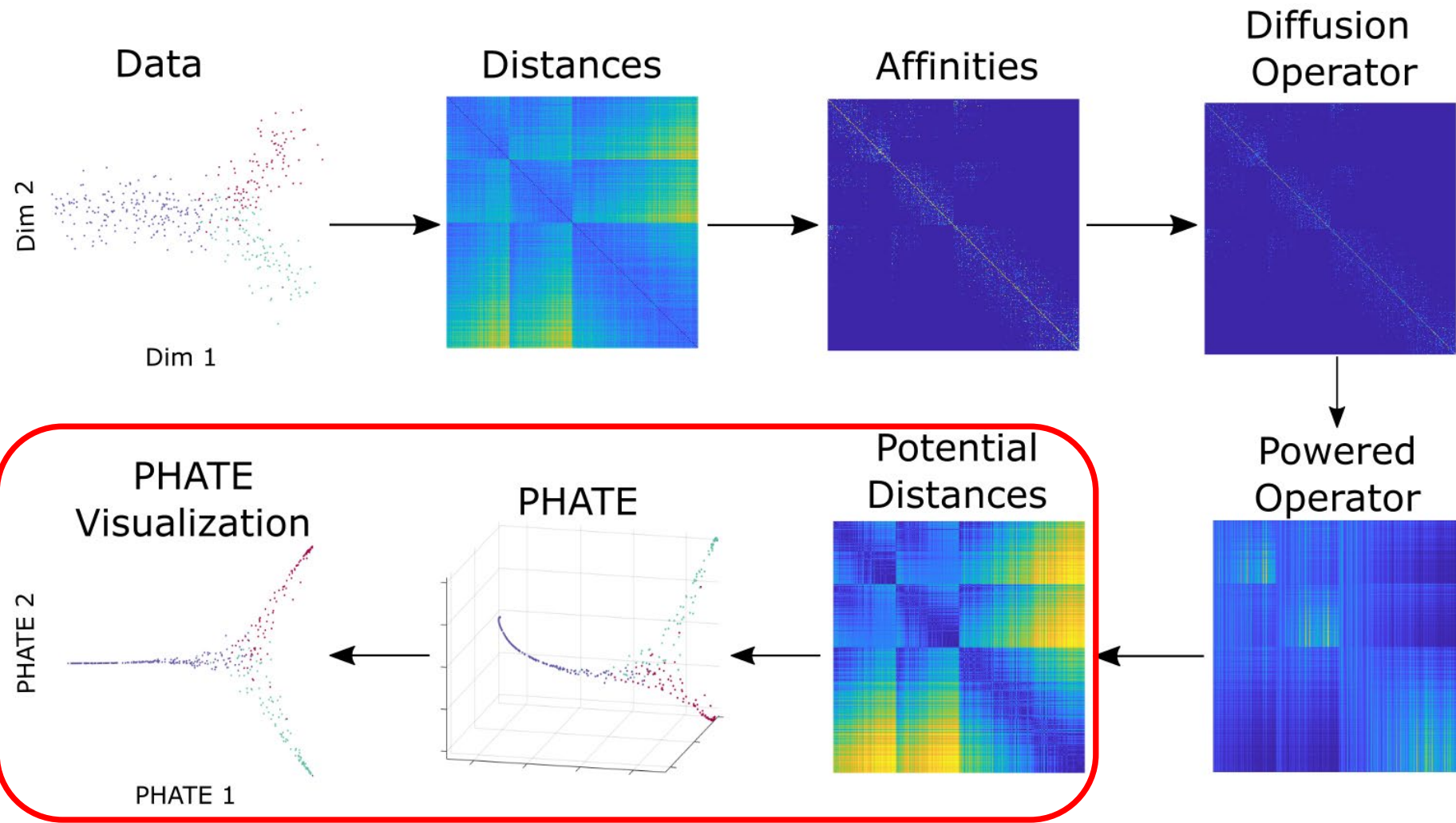
- VNE is a soft proxy of numerical rank
 - Decays as diffusion time increases, $\lim_{t \rightarrow \infty} VNE(P^t) = 0$
 - Use rate of decay to choose time scale
-

HOW



Embedding is
visually robust
in the flatter
region

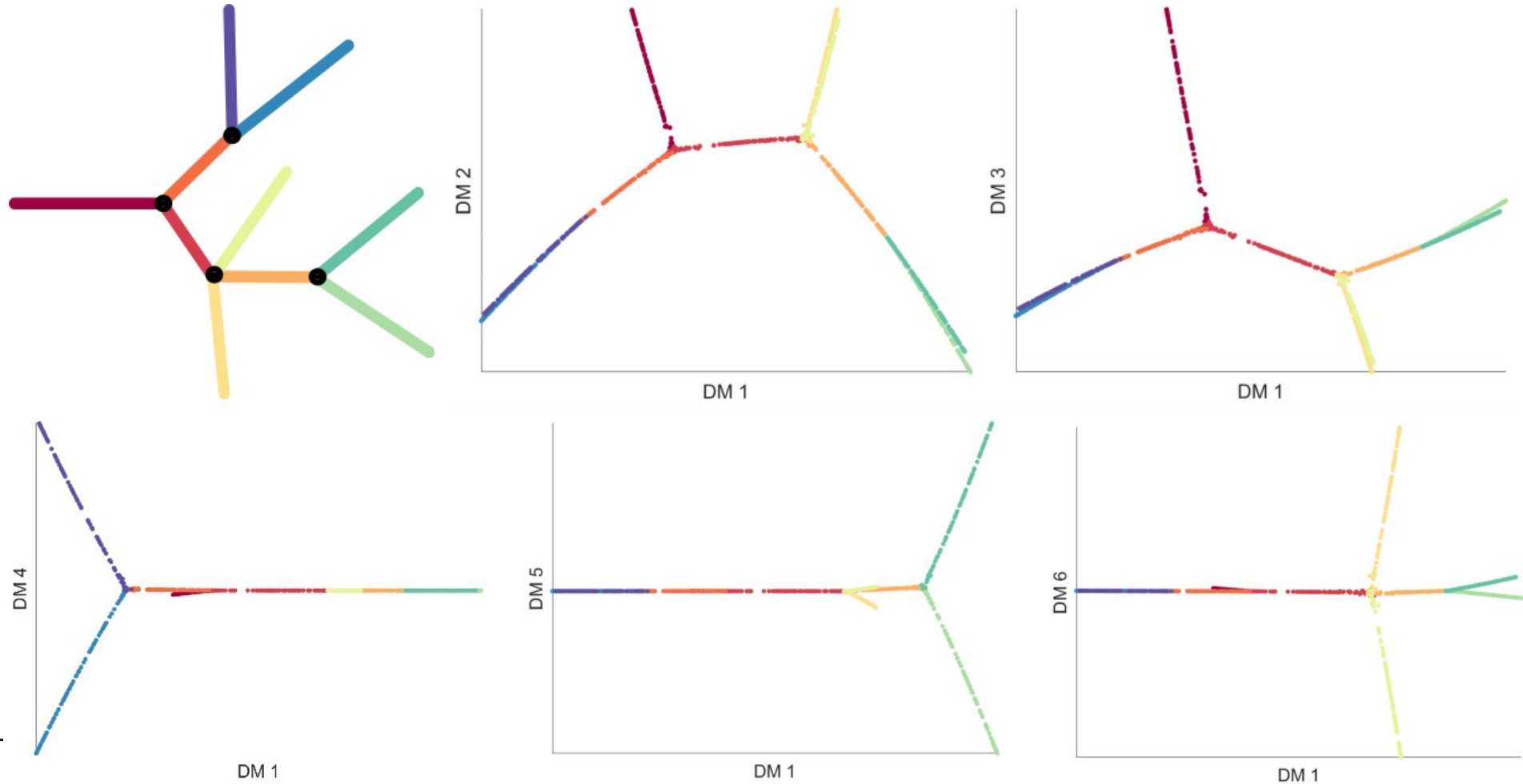
THE PHATE ALGORITHM



Transform to low
dimensions

POTENTIAL DISTANCES

- Diffusion operator contains global and local structure
 - Encoded in multiple dimensions



POTENTIAL DISTANCES

- Diffusion operator contains global and local structure
 - Encoded in multiple dimensions
- To extract this information, we transform diffused probabilities using a *potential transformation*
 - Forms an information distance between diffused probabilities
 - Connected to heat potential
- Embed for visualization using multidimensional scaling (MDS)

